Videolezioni — L07: Strutture dati complesse e iteratori

Sette video, **1 ora e 41 minuti in tutto**, uno per ciascun deck di lucidi. Si guardano **prima** della lezione in aula, dopo quelli di `L06`: per visitare una struttura si usa il `while`, e da qui arriva il `for`.

I video stanno nella cartella **Videolezioni** su Google Drive, in `L07_StruttureDati/`:

https://drive.google.com/drive/folders/1V1L8G9y0Rp7lQDawD49cLpxYVbG995hD

Si apre con l'**account campus**: è condivisa con i soli account d'Ateneo, e il link è anche sulla pagina Moodle nella sezione di questa lezione.

Video	Durata	Lucidi	Di cosa parla
`L07_a_StruttureDatiIntro`	8'56"	`L07_a`	che cos'è una struttura dati, e le proprietà che le distinguono: aggregazione, plasticità nel tempo, omogeneità; i tipi primitivi
`L07_b_StruttureDatiStringhe`	18'38"	`L07_b`	la stringa come sequenza di caratteri: indici, indici negativi, lunghezza, immutabilità, slicing, concatenazione e ripetizione, i metodi
`L07_c_StruttureDatiTuple`	14'53"	`L07_c`	la tupla: una sequenza eterogenea e immutabile; a che cosa serve, la virgola che la definisce, i metodi `index` e `count`
`L07_d_StruttureDatiListe`	17'47"	`L07_d`	la lista: una sequenza che si può modificare; `append`, `extend`, `del`, `sort`, e che cosa succede quando si copia una lista

Video	Durata	Lucidi	Di cosa parla
`L07_e_StruttureDatiDizionari`	7'48"	`L07_e`	il dizionario: valori raggiunti per chiave invece che per posizione; `keys`, `values`, `items`
`L07_f_StruttureDatiIteratori`	14'48"	`L07_f`	visitare una struttura: con il `while` e un indice, poi con il `for`; gli iteratori, e `range`
`L07_g_StruttureDatiMultidim`	18'41"	`L07_g`	le liste di liste: una tabella di dati, la somma per riga e per colonna con due cicli annidati; le strutture di strutture

L'ordine è quello della tabella. Ogni video segue il suo deck slide per slide, ma i lucidi sono stati rifatti in 16:10, con le sezioni in testa a ogni slide: se il video dice «slide 12», cercala per titolo, non per numero.

Come usarli assieme al resto del materiale

Se vuoi…	Guarda	Leggi
capire che cosa distingue una struttura dall'altra	`a`	`L07_a`, *Proprietà*; e la tabella finale di `L07_f`
lavorare sui caratteri di una stringa	`b`	`L07_b`, *Slicing* e *Metodi*
scegliere fra tupla e lista	`c`, `d`	`L07_c`, *Uso*; `L07_d`, *Liste*
capire perché modificare una copia cambia l'originale	`d`	`L07_d`, *Copia*
raggiungere un valore per nome	`e`	`L07_e`
scrivere il tuo primo `for`	`f`	`L07_f`, *for* e *range*
lavorare su una tabella di dati	`g`	`L07_g`, *Per riga* e *Per colonna*

Dove i lucidi e i video non coincidono

Questa è la sezione da leggere. I video sono stati registrati con Python 2, e i lucidi sono stati corretti dove il codice o un'affermazione non valgono più in Python 3. Nei video gli errori ci sono ancora: **dove le due cose divergono, valgono i lucidi.** In ordine di importanza:

`L07_g` — la tabella dei seggi era spostata di una colonna

Il caso di studio sono i voti delle elezioni presidenziali del 2000 in Florida, contea per contea. Ogni riga ha 17 valori, ma l'intestazione nel video ne nomina solo 16: manca il terzo campo, che vale 1 o 2 ed è il **numero di colonne della scheda elettorale**. Da lì in poi ogni valore finisce sotto il nome sbagliato: nel video Bush prende 105 voti ad Alachua e Gore 34124, mentre sono 34124 e 47365.

Anche i due programmi ne risentono, e **girano senza errori e rispondono male**: il totale per candidato attribuisce a «Bush» le schede nulle e a «Gore» i voti di Bush, e il totale per seggio somma il numero di colonne e perde gli ultimi tre candidati.

Sui lucidi la colonna si chiama `colonneScheda`, e i due programmi saltano i primi tre campi con una variabile dal nome esplicito:

```
primoVoto = 3   # salta Seggio, Strumento, colonneScheda
```

Su tutte le 67 contee, il programma corretto dà Bush 2911215 e Gore 2911417. È il tipo di errore più pericoloso: nessun messaggio, numeri plausibili, risposta sbagliata. Contare i campi di una riga e confrontarli con l'intestazione è il primo controllo da fare su qualunque tabella.

Sono corretti anche tre nomi di candidati: Phillips, Choate, McCarthy.

`L07_c` — `print` con più argomenti non stampa una tupla

Poco prima dei metodi delle tuple, il video mostra una slide intitolata **«Avrete notato…»**, che dice che

```
print("testo", 5, "altro testo", variabile)
```

stampa `('testo', 5, 'altro testo', 10)`, perché «l'insieme degli argomenti della `print()` viene interpretato come una tupla». **In Python 3 non è vero**: stampa

```
testo 5 altro testo 10
```

come vedi in ogni programma da `L04` in poi. Era vero in Python 2, dove `print` non era una funzione e le parentesi costruivano una tupla. **Sui lucidi quella slide non c'è più.** La parte vera — che è la virgola a fare la tupla — è nella slide *Attenzione!* con `b = 2,`.

`L07_d` — `input()` restituisce una stringa

Nelle slide di `append`, `extend` e `sort`, il video legge i numeri con `input(...)` e il `sort` li ordina come numeri. In Python 3 `input()` restituisce **sempre una stringa**, e con gli stessi valori del video il `sort` darebbe

```
['3', '32', '431', '6', '7381']
```

cioè l'ordine **alfabetico**: `'32'` viene prima di `'6'` perché `'3'` viene prima di `'6'`. Sui lucidi c'è `int(input(...))`, e il `sort` stampa `[3, 6, 32, 431, 7381]`.

`L07_f` — il `for` su un dizionario segue l'ordine di inserimento

Nella slide *Iteratori - ciclo for*, il video mostra le chiavi di un dizionario che escono in disordine (`four`, `five`, `three`, `two`, `one`) e conclude che **«l'insieme iteratore non ha un ordinamento»**. Da Python 3.7 non è più così: un dizionario ricorda l'ordine in cui le chiavi sono state inserite, e il `for` le visita in quell'ordine, `one`, `two`, `three`, `four`, `five`. Sui lucidi le uscite sono quelle di Python 3 e la conclusione è corretta.

Nella stessa slide il video dice che «viene creata una lista contenente le chiavi». In Python 2, con cui il video è registrato, era così; da Python 3.0 `diz.keys()` restituisce una **vista**, `dict_keys([...])`, non una lista. È la «vista» della tabella finale del deck. Per il `for` non cambia niente; se invece ti serve proprio la lista delle chiavi, per esempio per raggiungerle con un indice, basta convertirla:

```
listaChiavi = list(diz.keys())
```

È quello che fa la slide *Dizionari e visita dei loro elementi*, quella con il `while` e l'indice.

`L07_f` — `range` non è una lista, e `stop` è escluso

Nella slide su `range` il video dice che è «l'istruzione che restituisce una lista di interi» e che `stop` è «il valore massimo». La lista era vera in Python 2; è cambiata anche lei con Python 3.0. Sui lucidi:

- `range()` è una **funzione** che restituisce un oggetto iterabile, non una lista: `print(range(5))` stampa `range(0, 5)`, non `[0, 1, 2, 3, 4]`. Se ti serve la lista, anche qui basta convertirla: `list(range(5))`;
- `stop` è il primo valore **escluso**: `range(0, 100, 3)` arriva a 99. È anche l'unico argomento necessario: `start` vale 0 e `step` vale 1 se non si indicano.

`L07_a` — il tipo `long` non esiste più

Fra i tipi primitivi il video elenca `long`, «numero intero di lunghezza illimitata», e lo mostra con `type(1L)`. È un tipo di Python 2: in Python 3 c'è solo `int`, ed è `int` ad avere lunghezza illimitata. `1L` dà `SyntaxError`. Sui lucidi `long` non c'è, e l'ultima slide mostra `type(2 ** 100)`, che resta `<class 'int'>`.

`L07_g` — un dizionario si scrive con le chiavi

Nella slide *Strutture di strutture* il video scrive un dizionario di tuple come `diz = {tupla1, tupla2, ...}`. Con le graffe e senza i due punti, Python costruisce un **insieme**, non un dizionario, e non segnala niente. Sui lucidi: `diz = {chiave1: tupla1, chiave2: tupla2, ...}`.

Che cosa cambia nelle schermate del terminale

Molte slide mostrano schermate del terminale prese con Python 2. **Le uscite restano giuste**, ma il codice che vi compare va letto con due differenze:

- **`print` vuole le parentesi.** Dove la schermata mostra `print a` o `print lista[i]`, in Python 3 si scrive `print(a)`, `print(lista[i])`; senza parentesi è `SyntaxError`. Nei listati dei lucidi è già corretto;
- **i dizionari conservano l'ordine.** Dove una schermata stampa un dizionario o le sue chiavi in un ordine mescolato, Python 3 le stampa nell'ordine in cui sono state inserite.

Altre tre correzioni, più piccole, che nei video non ci sono:

- in `L07_c`, `b = 2,` stampa `(2,)`, con la virgola: il video mostra `(2)`. E `type` risponde `<class 'tuple'>`, non `<type 'tuple'>`;
- in `L07_f`, la slide che visita un dizionario con un indice usa `list(diz.keys())`: con `diz.keys()` da solo, in Python 3, `listaChiavi[i]` dà `TypeError`;
- in `L07_g`, le calorie del tiramisù si stampano senza parentesi: `Calorie totali tiramisu 4178.6`, non `('Calorie totali tiramisu', 4178.6)`.